1 Local Reduction Suffices for Universal Computation

Cellaria: A Model of Computation by Local Reduction

1.1 Abstract

We present Cellaria, a computational model based solely on local reduction: the state of a two-dimensional grid changes exclusively through rule-governed substitution of locally connected cell groups. The model has no processor, no bus, no shared memory, no instruction pointer, and no global control flow. Moving data requires a chained shift operation (a cell is copied to a distant location, then erased at its origin); competition between rules is resolved by greedy arbitration with priority and cell age.

We prove Turing completeness by two independent paths:

1. **Direct Turing machine simulation:** a constructive algorithm translates any Turing machine into a Cellaria rule set.
2. **Tag system reduction:** a tag system (Minsky, m=2) is simulated in Cellaria; tag systems are Turing-complete.

Both proofs rely strictly on the five axioms of Cellaria: homogeneous grid, computation through rules only, interface through boundary only, rules stored outside the grid, and cleanup through rules.

1.2 1. Introduction

1.2.1 1.1. What is Local Reduction?

Local reduction is a principle: the global state of a system changes only by replacing locally connected groups of elements according to fixed rules. No element has global authority. No central scheduler dispatches instructions. The computation is the collective effect of independent local decisions.

This principle appears in nature (chemical reactions, protein folding, crystal growth), in distributed systems (wireless sensor networks, gossip protocols), and in theoretical computer science (interaction nets, P-systems, cellular automata). However, existing models either impose global uniformity (cellular automata) or require complex graph rewriting (interaction nets).

Cellaria aims to be a minimal, concrete model of local reduction with deterministic semantics and practical implementation.

Formally, by “local reduction” in Cellaria we mean: (1) pattern matching over finite horizontal sequences of adjacent cells of arbitrary length, (2) asymmetric chained shift where only the center cell (the head) moves a finite number of steps in one direction and its origin is cleared to default; this preserves *locality of effect* (only two cells are modified) while relaxing the classical *locality of speed* constraint to permit distant jumps within a single tick, (3) priority-based greedy arbitration resolving overlapping matches deterministically.

This combination distinguishes Cellaria from: - cellular automata (fixed neighborhood, uniform rule), - interaction nets (graph rewriting with port connections), - string rewriting systems (no spatial embedding).

1.2.2 1.2. Why Not a Cellular Automaton?

Cellular automata (CA) are the most famous local computation model. Every cell updates by the same rule applied to a fixed neighborhood. Cellaria differs fundamentally:

- **Multiple rules:** Cellaria has many rules, not one. Rules compete.
- **Dynamic neighborhood:** pattern matching is not fixed to von Neumann or Moore neighborhoods — any 1D horizontal pattern is allowed.
- **Chained shift:** data moves by chained shift, not by copying to a neighboring cell. The original cell is cleared.
- **Greedy arbitration:** overlapping pattern matches are resolved by priority, age, and coordinates.

Cellaria is closer to a rule-based rewriting system embedded in a grid than to a cellular automaton.

This distinction is practically relevant. The proof of Turing completeness for the elementary cellular automaton Rule 110 [Cook 2004] relies on demonstrating that complex emergent structures (gliders, collisions) simulate a cyclic tag system — a tour de force of global analysis. In Cellaria, by contrast, the translation from a Turing machine is direct, modular, and constructive: each TM transition becomes exactly one Cellaria rule, with Lemma 1 guaranteeing conflict-free execution. This makes Cellaria programs amenable to compositional verification, a property notoriously difficult for cellular automata.

1.2.3 1.3. Why Not Turing Machines?

A Turing machine has a central head and a tape. In Cellaria, both head and tape are cells on the same grid. There is no architectural separation of control and memory. The head is just a cell with a special type; movement is chained shift, not a head moving over a static tape.

The Turing machine is our target for the completeness proof, not our inspiration.

1.2.4 1.4. Related Work

Model	Key difference from Cellaria
Cellular automata	Single rule, fixed neighborhood
Interaction nets [Lafont 1990]	Graph rewriting, port connections
P-systems [Păun 1998]	Hierarchical membranes
String rewriting (Post, Thue)	No spatial structure
Chemical abstract machine [Berry & Boudol 1992]	Multiset rewriting, no geometry
MGS (spatial computing) [Giavitto 2002]	Topological collections, declarative
Amorphous computing [Abelson 2000]	No fixed grid, probabilistic
Shape computing	Pattern matching on arbitrary neighbourhoods
Cell programming language [Shapiro 1995]	Cellular automata variant
GP 2 [Plump 2012]	Graph programs, not grid-based

Cellaria occupies a specific niche: grid-based, multi-rule, priority-driven, with chained shift as the only movement mechanism. Unlike MGS, Cellaria does not require topological collections; unlike amorphous computing, it is deterministic and operates on a fixed grid.

1.3 2. The Cellaria Model

1.3.1 2.1. Five Axioms

The model is defined by five axioms. Any claim about Cellaria must respect these constraints.

Axiom 1: Homogeneous Grid. No cell has privileged status by coordinate. No reserved system zones. Channels (I/O boundaries) are permitted because they are external to the grid, not internal privileges.

Axiom 2: Computation Through Rules Only. All state changes inside the grid happen through the detect → arbitrate → apply cycle. No hidden mechanism modifies cells without rule participation. Cell age is passive metadata, it influences arbitration but never changes state directly.

Axiom 3: Interface Through Boundary Only. Input and output transport data across the system boundary. They are not computation. Data crossing the boundary is I/O; data changing inside the grid is computation. The boundary phases (input, flush) do not compete with rules.

Axiom 4: Rules Outside the Grid. Rules are stored externally and updated atomically between ticks. Storing rules inside the grid would require reserved zones (violating Axiom 1) or risk races (rules changing mid-tick).

Axiom 5: Cleanup Through Rules. If a cell should disappear, a rule must do it. The `min_age` field in rules enables time-based cleanup: a rule activates only if its center cell has not changed for N ticks. No hidden garbage collection timer exists.

1.3.2 2.2. Grid and Cells

The grid is two-dimensional and rectangular. Each cell stores:

- A type value `t ∈ {0..255}`, where `t = 0` is the default (empty) type
- An age counter, incremented every tick and reset when the cell changes

1.3.3 2.3. Rules

In the current implementation, a rule is defined by:

- `id`: a `RuleId` (a sequence of `CellType` values, `Vec<CellType>`). The first element `id[0]` is the center (the head). The head determines the matching position and is the only cell that moves on shift.
- `pattern`: reserved for future use (`Vec<Vec<u8>>`, currently unused).
- `shifts`: zero or more shift groups. Each group is a sequence of directional shifts (east, west, up, down) applied to the head. Groups are executed in priority order.
- `changes`: post-shift modifications `(dx, dy, value)` applied relative to the head's post-shift position. **Changes are applied only if at least one shift was executed.** A shift of 0 steps in any direction is treated as an executed shift (not used in the constructions of this paper, but supported by the model).
- `priority`: higher priority rules win in conflicts.
- `min_age`: minimum age of the center cell for the rule to activate.
- `active_only`: when true, matching is restricted to neighborhoods of non-default cells (optimization).

For the formal definition of the data structures, see the specification (`specs/specification.md`, Section 3). The current implementation includes additional fields (notably `pattern`, reserved for

future 2D pattern matching extensions). For the formal model and all proofs in this paper, only the fields described above are relevant.

1.3.4 2.4. Chained Shift

Chained shift is the only data movement mechanism. When a rule with a shift matches:

1. The head cell `(cx, cy)` is copied `steps` cells in the given direction.
2. If the destination is within the grid, the head is copied there.
3. If the destination is outside, the head enters an output buffer.
4. The original position `(cx, cy)` is cleared to default (0).

Only the head moves. The rest of the pattern (the "tail") stays in place. This is asymmetrical: the head is the active element; the tail is passive context.

Remark (Locality of effect vs. locality of information speed). A chained shift with `steps > 1` changes only two cells: the destination cell (which receives the head's type) and the origin cell (which is cleared). In this sense, the *effect* of the rule is strictly local: it touches only the endpoints of the shift. This is distinct from *locality of information speed* in classical cellular automata, where each cell can only influence its immediate neighbors per tick and information propagates at speed 1. Cellaria intentionally relaxes the speed constraint while preserving locality of effect: no rule inspects or modifies the intermediate cells between origin and destination. Theorem 1 then demonstrates that even with this relaxed notion of locality — where information can leap — no global control is required for universal computation. Characterising the exact boundary between locality of effect and locality of speed remains an open theoretical question.

On the term "chained". The term "chained" refers to the fact that multiple shift groups within a single rule are executed sequentially in priority order, each group potentially moving the head further. A single shift within a group is an atomic jump from origin to destination; it does not chain through or inspect intermediate cells.

Interaction with pattern tail. When a rule with pattern `[HeadType, TailType]` and a shift of 1 step is applied, the sequence is:

1. The head cell (center, `HeadType`) is copied to the destination cell. The original head position is cleared to 0.
2. The destination cell — which previously held `TailType` — is overwritten by the incoming `HeadType`. The tail is thus consumed by the head's movement.
3. The `changes` phase then writes a new tail symbol at the position vacated by the head (now 0), using coordinates relative to the head's post-shift position.

This guarantees atomicity: the new head and new tail appear simultaneously, and no cell is ever simultaneously claimed by two operations. The old tail is naturally overwritten by the incoming head rather than being explicitly cleared, because the shift phase copies the head *onto* the tail's position.

1.3.5 2.5. Tick Cycle

Each simulation tick has five phases:

1. **Input:** data from input buffers is written to boundary cells.
2. **Detect:** scan the grid for pattern matches.
3. **Arbitrate:** resolve conflicts by priority, age, coordinates.
4. **Apply:** execute shifts and changes for accepted matches.

5. **Flush:** collect output, advance ages.

1.3.6 2.6. I/O Boundaries

Boundary cells are on the grid edge. Input buffers feed data to boundary cells at the start of each tick. When a cell shifts out of the grid, it enters an output buffer. Buffers are organized by channel number.

1.4 3. Proof 1: Direct Turing Machine Simulation

1.4.1 3.1. Formal Definitions

Definition 1 (Encoding function). Let a Turing machine be given as

$M = (Q, \Gamma, \delta, q_0, q_h)$

where $Q = \{q_0, q_1, \ldots, q_{n-1}, q_h\}$ is the set of states, $\Gamma = \{\gamma_0, \gamma_1, \ldots, \gamma_{k-1}\}$ is the tape alphabet with blank symbol $\square \in \Gamma$, and $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, H\}$ is the transition function.

We define the encoding function $\mathrm{enc} : (Q \cup \Gamma) \to \mathrm{CellType}$:

$$\begin{aligned}
&\mathrm{enc}(\gamma_i) = i + 1 && \text{for } \gamma_i \in \Gamma,\ \gamma_i \neq \square \\
&\mathrm{enc}(\square) = 0 && \\
&\mathrm{enc}(q_i) = 10 + i && \text{for } q_i \in Q,\ q_i \neq q_h \\
&\mathrm{enc}(q_h) = 10 + |Q| && \text{(halt state marker)}
\end{aligned}$$

Since $\mathrm{enc}(\square) = 0$ and type 0 is the default (empty) cell type in Cellaria, blank tape cells are naturally encoded as empty grid cells. This alignment is intentional: it ensures that any uninitialised region of the grid behaves as a blank tape, and no explicit blank-filling is required.

The encoding function enc is injective on $Q \cup \Gamma$: distinct states and tape symbols map to distinct cell types. This ensures that every TM configuration has a unique Cellaria representation.

Definition 2 (Cellaria grid configuration, revised). A 1×N grid G encodes a TM configuration $(q, \mathrm{tape}, \mathrm{pos})$ such that:

- The head marker $\mathrm{enc}(q)$ occupies grid position pos.
- The tape symbol $\mathrm{enc}(\mathrm{tape}[\mathrm{pos}])$ occupies grid position $\mathrm{pos}+1$.
- All other tape symbols occupy their respective positions.

Invariant: The head marker is always immediately left of the current tape symbol. This ensures that the pattern $[\mathrm{enc}(q), \mathrm{enc}(\mathrm{tape}[\mathrm{pos}])]$ always matches at position pos with the center at pos and $\mathrm{id}[1]$ at $\mathrm{pos}+1$.

For the encoding to be valid, the grid must have size at least $\max(\mathrm{pos} + 1)$ where pos is the head position. Blank cells ($\mathrm{enc}(\square) = 0$) may extend beyond the tape to fill the remainder of the grid. This ensures that the rule $[\mathrm{enc}(q), \mathrm{enc}(a)]$ always has a complete pattern match at pos.

For a move left transition $\delta(q, a) = (q', a', L)$, after the shift: - The head marker moves from pos to $\mathrm{pos}-1$. - The current symbol a' is written at pos (vacated by head). - The symbol at $\mathrm{pos}-1$ is now the new “current” symbol from $\mathrm{tape}[\mathrm{pos}-1]$. - The new pattern $[\mathrm{enc}(q'), \mathrm{enc}(\mathrm{tape}[\mathrm{pos}-1])]$ matches at $\mathrm{pos}-1$.

This ensures the invariant is maintained after each step.

The proof uses a 1×N grid for simplicity. Extension to the full 2D grid is straightforward: the tape occupies row 0, and additional rows may be used for auxiliary computation or multi-tape simulations without affecting the construction.

Definition 3 (Rule construction, revised notation). For each transition `δ(q, a) = (q', a', d)`, construct a Cellaria rule `R(q, a)`:

```
R(q, a) = Rule(
    id = [enc(q), enc(a)],          // head marker + tape symbol
     shifts = [ [shift_spec(d)] ],   // one shift group
   changes = change_spec(d, enc(a')),  // returns a list of (dx, dy, valu
     priority = 10,
     min_age = 0,
     active_only = false
)
```

where:

```
shift_spec(L) = { direction: west, steps: 1 }
shift_spec(R) = { direction: east, steps: 1 }
shift_spec(H) = { direction: east, steps: 1 }  // standard rightward shi
                                               // for uniform handling

// change_spec(d, val) returns a list of (dx, dy, value) tuples.
// For move transitions the list has one element; for halt it has two.
change_spec(R, val) = [(-1, 0, val)]                  // head moved right
                                                      // vacated cell is at dx = -
1
change_spec(L, val) = [(+1, 0, val)]                  // head moved left (
                                                      // vacated cell is at dx = +1
change_spec(H, val) = [(-1, 0, enc(q_h)), (0, 0, val)]   // at vacated p
                                                      // at new head position: write
                                                      // both elements applied atomi
                                                      // in the same changes phase
```

The change_spec coordinates are relative to the head's post-shift position. For a rightward move, the vacated cell is one step to the left of the new head position (dx = -1). For a leftward move, the vacated cell is one step to the right of the new head position (dx = +1).

For the halt transition, the rule uses a standard rightward shift of 1 step. Two changes are applied: at the vacated position (dx = -1), the halt marker `enc(q_h)` is written, replacing the original head; at the new head position (dx = 0), the updated tape symbol `val` is written, replacing the previous tape symbol. After this tick, the grid contains `enc(q_h)` immediately left of the updated tape symbol, maintaining the invariant from Definition 2. On the next tick, the detect phase finds no rule matching `enc(q_h)`, and the system stabilises.

By Definition 1, `enc(q_h) = 10 + |Q|`. Since no rule is generated with `id[0] = enc(q_h)`, the head marker for the halt state has no matching rule. The system stabilizes when the head enters this state.

Definition 4 (Initial configuration). Given TM input `w = a□a□...a_{m-1}`, the initial Cellaria

grid is:

```
G[0] = enc(q□)          // head marker
G[1] = enc(a□)          // first symbol
G[2] = enc(a□)
...
G[m] = enc(a_{m-1})
G[m+1..N-1] = 0         // blank tape
```

1.4.2 3.2. Theorem 1 (Turing Completeness of Cellaria)

Theorem 1. For any Turing machine M and any input w, there exists a finite Cellaria rule set R and a finite grid G such that for all n □ □:

> M accepts w in n steps (the n-th step enters the halting state q_h) □
> Cellaria with rule set R executes exactly n computational ticks (each corresponding to one TM transition), after which the grid contains enc(q_h) as the head marker. On tick n+1, the detect phase finds zero matches and the system stabilises with no further state changes. The number of computational ticks equals the number of TM steps.

Proof. We prove by induction on the number of steps n.

Base case (n = 0). The initial TM configuration M □□ (q□, w, 0) maps to the initial Cellaria grid G□ per Definition 4. No rule has applied yet, matching the TM state before any step.

Inductive step. Assume that after n steps, the TM configuration (q, tape, pos) maps to Cellaria grid G□ per Definition 2, and that no rule has been applied in tick n (the tick counter is at n, awaiting the next tick).

We show that one tick of Cellaria (phases detect → arbitrate → apply) produces grid G_{n+1} that corresponds to the TM configuration after step n+1.

Step 1 (detect). The grid G□ contains pattern [enc(q), enc(tape[pos])] at position pos. The rule R(q, tape[pos]) is present in R by Definition 3. The detect phase finds this match.

Step 2 (arbitrate). Only one rule matches at position pos because enc(q) is unique per state. Lemma 1 (below) guarantees no overlapping matches. The rule is accepted.

Step 3 (apply). According to the Cellaria tick cycle (Section 2.5), shifts are applied before changes.

1. **Shift phase:** head cell at (cx, cy) is copied to (cx', cy'), then the original position (cx, cy) is cleared to default (0).
2. **Changes phase:** for each (dx, dy, value) in changes, a cell at (cx' + dx, cy' + dy) is set to value.

For TM rules, the changes specification is change_spec(d, enc(a')) as defined in Definition 3. Since shift has already cleared (cx, cy) before changes execute, the write is atomic and unambiguous.

After application, the grid state is:

- If `d = R`: `G_{n+1}[pos] = enc(a'), G_{n+1}[pos+1] = enc(q')`, and `G_{n+1}[pos+2] = enc(tape[pos+1])`. This corresponds to TM configuration `(q', tape[pos←a'], pos+1)`.
- If `d = L`: after shift, the head moves from `pos` to `pos-1`. The `changes` specification `change_spec(L, enc(a')) = (+1, 0, enc(a'))` writes `enc(a')` at `dx = +1` relative to the new head position, i.e., at `pos` (the cell vacated by the head). Thus: `G_{n+1}[pos-1] = enc(q'), G_{n+1}[pos] = enc(a')`. The invariant holds: `enc(q')` is now immediately left of `enc(tape[pos-1])` at their respective positions, forming the pattern `[enc(q'), enc(tape[pos-1])]` at the new head position `pos-1`. This corresponds to TM configuration `(q', tape[pos←a'], pos-1)`.
- If `d = H`: the rule performs a standard rightward shift of 1 step. The head moves from `pos` to `pos+1`. Two changes are applied: `change_spec(H, enc(a'))[0] = (-1, 0, enc(q_h))` writes the halt marker `enc(q_h)` at the vacated position `pos` (dx = -1 relative to the new head position `pos+1`). `change_spec(H, enc(a'))[1] = (0, 0, enc(a'))` writes the updated tape symbol `enc(a')` at the new head position `pos+1`. Thus: `G_{n+1}[pos] = enc(q_h), G_{n+1}[pos+1] = enc(a')`. The invariant from Definition 2 is maintained: the head marker `enc(q_h)` is immediately left of the current tape symbol. Since no rule has `id[0] = enc(q_h)` (by Definition 1), the detect phase of the next tick finds zero matches, and the system stabilises.

Halt. If `δ(q, a) = (q', a', H)`, the transition is executed in one computational tick as described above (case d = H), producing a grid with head marker `enc(q_h)`. On the next tick (n+1), the detect phase finds zero matches, and the system stabilises. The halt transition itself is a regular computational step; stabilisation is the absence of further computation.

Thus, each Cellaria tick corresponds to exactly one TM step, and the sequences of configurations are isomorphic. □

1.4.3 3.3. Lemma 1 (Conflict-Free Rules)

```
Lemma 1. For a Turing machine M with state set Q,
let R be the rule set constructed per Definition 3.
Then for any two distinct rules r□, r□ □ R,
there is no grid state g where both r□ and r□ match
at overlapping cells.
```

Proof. Two rules `r□` and `r□` may only conflict if their patterns overlap at the same grid position. The matching pattern of any rule `R(q, a)` is `[enc(q), enc(a)]`, where `enc(q)` is the head marker and `enc(a)` is the tape symbol.

Consider two cases:

Case 1: q□ ≠ q□. Then `enc(q□) ≠ enc(q□)` by Definition 1 (since `enc` is injective on `Q`). The first element of `id(r□)` differs from the first element of `id(r□)`. Pattern matching requires `id[0]` to match the center cell. Since `enc(q□) ≠ enc(q□)`, no cell can be the center of both patterns simultaneously.

Case 2: q□ = q□ = q but a□ ≠ a□. Then `enc(a□) ≠ enc(a□)` (the encoding is injective on `Γ`). The patterns are `[enc(q), enc(a□)]` and `[enc(q), enc(a□)]`. They share

the same center cell type `enc(q)`, but the second cell `id[1]` differs. For both to match at the same position, the cell to the right of the center would need to be both `enc(a□)` and `enc(a□)`, which is impossible.

Case 3: q□ = q□ and a□ = a□. Then `r□ = r□` (identical transition). The rule set may contain duplicate entries, but they are logically identical and produce the same match. No conflict arises.

Thus, no two distinct rules can match at overlapping cells. □

Remark (Extension to dynamic patterns). While the constructed TM rule set uses only two-element patterns `[enc(q), enc(a)]`, Cellaria's rule syntax permits single-element patterns `[enc(q)]` and longer patterns. For completeness, we note that Lemma 1 generalizes: any rule set where patterns are disjoint (no two patterns share the same cell type at matching positions) will have conflict-free execution. This enables future extensions (e.g., multi-head rules, variable-length patterns).

Corollary 1. Under Lemma 1, arbitration is deterministic: for any grid state, at most one rule matches at each position, and no two matches overlap. The greedy arbitration algorithm accepts all matches.

Corollary 2 (Parallel execution). Under the TM rule set constructed per Definition 3, Lemma 1 guarantees that no two rules match at overlapping positions. Therefore, all matches in a given tick are pairwise independent and can be applied in any order — or simultaneously in a single parallel pass over the grid — without affecting the final configuration. This provides a direct path to efficient parallel implementation on GPU or FPGA architectures, as the detect and apply phases for the TM simulation require no sequential arbitration.

1.4.4 3.4. Implementation: Bit Inversion

The configuration `configs/turing.yaml` implements a bit-inverting TM with tape alphabet `Γ = {0, 1}` and a single state `Q□`:

```
head marker:        10   (enc(Q□) = 10)
tape symbol 1:      1    (enc(1) = 1)
tape symbol 0:      2    (enc(0) = 2)
blank:              0
```

Rules:

```
id=[10, 2] → shift east, change (-1, 0, 1)   read 0, write 1, move right
id=[10, 1] → shift east, change (-1, 0, 2)   read 1, write 0, move right
```

The change at `dx = -1` writes the new symbol at the position vacated by the head. The head type remains `10` (no state change), so the head continues moving right. When the head reaches a blank cell (type 0), no rule matches and the system halts.

The inverse encoding (type 2 = 0, type 1 = 1) is used to make the output symbols visually distinct from the input symbols, facilitating manual verification of correctness.

1.4.5 3.5. Complexity of TM Simulation

Theorem 2 (Time preservation). Each simulated TM transition, including the transition that writes the halting state `q_h`, is executed in exactly one Cellaria tick.

Proof. The inductive proof of Theorem 1 establishes a bijection between TM transitions and Cellaria ticks. The halt transition is a regular transition that writes `q_h`; the subsequent stabilisation (zero matches in the next detect phase) is not a computation step. □

Theorem 3 (Space efficiency). Simulating a TM with `n` states and tape length `k` requires `O(n + k)` Cellaria cells on a 1×N grid.

Proof. The tape requires `k` cells (one per tape position). The head requires 1 cell of type `enc(q)`, which is distinct from tape symbols. The total is `k + 1` cells. The number of states `n` affects the rule set size, not the grid size. The encoding `enc(q) = 10 + i` requires no additional grid cells. □

1.5 4. Proof 2: Reduction via Tag System

1.5.1 4.1. Tag Systems (Minsky 1961)

A tag system with deletion number `m = 2` and alphabet `Σ = {A, B}`:

- Configuration: a string `w` ∈ `Σ*`
- Step: let `X = w[0]`. Delete the first `m = 2` symbols from `w`. Append the production `π(X)` ∈ `Σ*` to the end.
- Halt: when `|w| < m`.

Minsky proved that tag systems with `m = 2` and binary alphabet are Turing-complete [Minsky 1961].

For our demonstration, we use productions `π(A) = AB`, `π(B) = A`.

1.5.2 4.2. Simulating One Tag Step in Cellaria

The configuration `configs/tag_system.yaml` implements one tag system step.

Grid layout (1×32):

```
[10, A, B, B, 0, 0, ...]
  ↑   ↑
marker  string
```

Marker states:

Type	Meaning
10	Start: read first symbol
11	Read A, deleting second symbol
12	Read B, deleting second symbol
13	Traverse (A→AB): shifting symbols left
14	Traverse (B→A): shifting symbols left
15	Write A (first symbol of AB production)
16	Write A (A production)
17	Write B (second symbol of AB production)

Execution phases:

1. Read and delete m=2: `[10, X, Y] → [0, 0, 13|14]`

2. Traverse: `[13|14, X] → [X, 13|14]` — compress the string leftward
3. Write production: `[13|14, 0] → [π(X), 0]` — production is written, marker disappears via change `[0, 0, 0]`. Tick stops because next tick finds zero matches.

For concreteness, the trace below demonstrates one tag step on the initial string A B B, which transforms to B A B under productions π(A) = AB, π(B) = A.

1.5.3 4.3. Trace Example (A B B → B A B)

From actual simulation:

```
Tick 0: [10, 1, 2, 2, 0, ...]   // initial: A B B (1=A, 2=B)
Tick 1: [0, 11, 2, 2, 0, ...]   read A
Tick 2: [0, 0, 13, 2, 0, ...]   delete second B
Tick 3: [0, 0, 2, 13, 0, ...]   carry B left
Tick 4: [0, 0, 2, 1, 15, ...]   write A (AB production)
Tick 5: [0, 0, 2, 1, 2, ...]   write B, marker changes to (0,0,0) → 0 matc
Tick 6: [0, 0, 2, 1, 2, ...]    stable: B A B □
        (no rule matches; system halts)
```

1.5.4 4.4. Composition and Axiom 3 (Revised)

One tag system step corresponds to one Cellaria run: from initial state to a stable configuration with no matching rules. The result is the string prepared for the next step.

Meta-Interpreter Construction:

To simulate multiple tag steps without external orchestration, we can construct a meta-interpreter within Cellaria itself:

A finite rule set `R_meta` that: 1. Reads an input string from boundary (Axiom 3, input phase). 2. Executes one tag step using rules from Section 4.2. 3. Upon completion (zero matches), signals readiness via output. 4. Waits for next input, then repeats.

The composition of meta-interpreter rules with boundary I/O rules creates a feedback loop: output from tick N becomes input for tick N+1. By Axiom 3, this is permitted because I/O is external to the grid.

Theorem 5 (Tag System Composition). A Cellaria meta-interpreter constructed as above can execute an arbitrary finite sequence of tag system steps by repeated boundary feedback without modification to the rule set.

Thus, any tag system computation (which is a finite sequence of steps, or a potentially infinite sequence that terminates) is realised by Cellaria with external coordination through Axiom 3.

We acknowledge that the meta-interpreter construction relies on boundary I/O (Axiom 3) for step composition. A fully internal tag system interpreter — where the entire computation cycle runs within the grid without external intervention — remains an open engineering challenge. However, this does not weaken the completeness result: Axiom 3 is part of the model, and external coordination is a legitimate use of it. This proof path demonstrates that tag system semantics are expressible in Cellaria; the direct TM simulation (Proof 1, Theorem 1) provides the stronger, fully internal completeness result.

Fully internal vs. I/O-assisted interpretation.

The meta-interpreter construction in this section achieves Turing completeness through boundary I/O feedback. An alternative, fully internal architecture is feasible: upon completion of a tag step (zero matches), an additional rule could detect this state and reinitialise the marker at position 0. Such a rule would match the pattern `[0, any]` at the grid start and write a new marker `enc(start)`, triggering the next step automatically.

This internal restart is a natural extension of Cellaria's capabilities (detection of empty cells, arbitrary writes via `changes`) and confirms that the meta-interpreter construction is not a fundamental limitation of the model, but rather a choice of implementation strategy.

Corollary 5.1. By Minsky's theorem, tag systems with `m = 2` are Turing-complete. Therefore, Cellaria is Turing-complete. *Note: this completeness path assumes external coordination via Axiom 3 for step composition; Theorem 1 provides the stronger, fully internal result.*

1.5.5 4.5. Transitive Completeness

1. Tag systems (m=2) are Turing-complete [Minsky 1961]; the binary encoding is historically discussed in Cocke and Minsky (1964).
2. One tag system step is simulated by Cellaria (Section 4.2).
3. Step composition is orchestrated through boundary I/O (Axiom 3). Therefore, Cellaria is Turing-complete.

1.5.6 4.6. Complexity of Tag System Simulation

Theorem 4 (Tag step complexity). One tag system step on a string of length `k` requires `O(k)` Cellaria ticks.

Proof. The simulation consists of three phases: - Read and delete m=2: 2 ticks. - Traverse: `k − 2` ticks (one symbol shifted left per tick). - Write production: `|π(X)|` ticks (one symbol per tick). Total ticks: `2 + (k − 2) + |π(X)| = k + |π(X)| = O(k).` □

1.6 5. Discussion

1.6.1 5.1. Two Independent Proofs

We have established Turing completeness by two independent paths:

```
Path 1: Turing machine → Cellaria (direct encoding, turing.yaml)
Path 2: Tag system → Turing machine [Minsky] → Cellaria (tag_system.yam
```

The first path is constructive and direct: any TM transition table produces a Cellaria rule set. The second path is indirect but simpler in execution: a tag system step requires only forward movement, no back-and-forth head motion. Together, they rule out systematic translation errors.

1.6.2 5.2. Limitations

- **Grid size:** Cellaria supports both fixed-size (`VecStorage`) and unbounded (`ChunkStorage`) grids. For both completeness proofs, a finite grid is sufficient: the Turing machine tape grows only within its initial allocation, and the tag system string fits within the 1×32 row. No boundary I/O is required for these demonstrations.

- **Single head:** Cellaria rules match a single head per pattern. Truly parallel multi-head computation is not demonstrated here.
- **Determinism:** greedy arbitration is deterministic, but the model permits non-deterministic rule sets (overlapping patterns with equal priority). The completeness proofs assume deterministic arbitration. Lemma 1 shows that the constructed TM rule sets are always conflict-free.

1.6.3 5.3. Comparison with Other Models

Interaction Nets (Lafont 1990): Interaction nets are graph-based and require port connections. Cellaria uses a homogeneous grid with geometric adjacency. Interaction nets have a stronger notion of locality (only active pairs interact); Cellaria rules can inspect longer patterns. Interaction nets are also Turing-complete, but their graph rewriting is more complex than Cellaria's chained shift.

P-Systems (Păun 1998): P-systems use hierarchical membrane structures with evolution rules. Cellaria has no hierarchy; all cells are equal. P-systems achieve Turing completeness through membrane dissolution and creation, which have no analogue in Cellaria.

Chemical Abstract Machine (CHAM, Berry & Boudol 1992): CHAM uses multiset rewriting with no spatial structure. Cellaria adds geometry: cells have coordinates, distance matters, and chained shift provides directional movement. CHAM's completeness relies on the ability to encode any multiset rewriting system; Cellaria's completeness is proved by explicit simulation.

MGS (Giavitto 2002): MGS is a spatial computing language based on topological collections. It supports multiple spatial structures (arrays, graphs, Delaunay triangulations) and declarative rewriting. Cellaria is more constrained (grid only, chained shift only) but simpler to analyze.

Amorphous Computing (Abelson et al. 2000): Amorphous computing models computation in irregular, probabilistic environments. Cellaria is deterministic and grid-based, making it more suitable for formal verification.

1.6.4 5.4. Complexity Analysis Summary

Metric	Path 1 (TM)	Path 2 (Tag system)
Ticks per step	1 (Theorem 2)	O(k) per tag step (Theorem 4)
Grid cells	O(k + n) (Theorem 3)	O(k)
Rules	O(	Q
Rules (demo)	O(	Q

1.6.5 5.5. Implications

The result "local reduction suffices for universal computation" has both theoretical and practical implications:

- Theoretically, it establishes a lower bound: even minimal locality (fixed pattern matching, chained shift, greedy arbitration) is sufficient for universality. No global control, no shared memory, no central scheduler is required.
- Practically, the Cellaria model maps naturally to hardware without shared buses: each cell can be an independent processing element communicating only with immediate neighbors. This is relevant for novel computing substrates (e.g., molecular computing, optical computing, distributed sensor networks).

- The model's determinism (Lemma 1) and the formal proof of conflict-free TM rules provide a foundation for verification of Cellaria programs.
- An additional implication is the model's potential for parallelism. Lemma 1 demonstrates that for the constructed TM rule sets, rules never conflict, and thus all matches can be applied simultaneously in a single tick. For general rule sets, Cellaria's semantics do not preclude parallel execution: any set of non-overlapping matches is guaranteed to produce the same result regardless of application order. However, the greedy arbitration algorithm as currently specified resolves overlaps sequentially by priority. Characterising exactly which rule sets admit fully parallel execution — and whether a static analysis can determine this — remains an open question.
- The relaxation of locality of speed, while preserving locality of effect, raises a fundamental question: what is the minimal speed of information propagation necessary for universality in a locally-acting system? Cellaria provides an upper bound: a single tick suffices for any finite jump.

1.6.6 5.6. Open Questions and Future Work

Several directions remain open:

1. **Non-determinism:** What is the expressiveness of Cellaria with non-deterministic rule sets (overlapping patterns with equal priority)?
2. **Optimization:** Can Cellaria programs be compiled to efficient machine code, e.g., for GPU execution?
3. **Abstraction:** What higher-level programming languages can be designed to translate naturally to Cellaria rules?
4. **Spatial complexity:** What spatial lower bounds exist for Cellaria programs? Can we prove that some computations inherently require Ω(n) grid cells?
5. **Variants:** What happens if we extend to 3D grids, or if we permit diagonal shifts?

1.7 6. Conclusion

We have presented Cellaria, a computational model based entirely on local reduction. The model is defined by five axioms that ensure locality: homogeneous grid, computation through rules only, interface through boundary only, rules stored externally, and cleanup through rules.

Two independent proofs demonstrate Turing completeness:

1. **Theorem 1** provides a *strong, fully internal* completeness result: the entire computation runs within the grid, requiring no external intervention or I/O beyond the initial configuration.
2. **Section 4** provides an *I/O-assisted* completeness proof via tag systems (Minsky, m=2). While step composition here relies on boundary I/O (Axiom 3), this proof path independently confirms that tag system semantics are faithfully expressible in Cellaria.

Both proofs rely solely on the primitive operations: pattern matching, chained shift, and greedy arbitration. No global control, no shared memory, and no central scheduler is required.

The result establishes that local reduction, as formalized by Cellaria, is a sufficient basis for universal computation. Moreover, this work demonstrates that local reduction is not merely sufficient but also efficient for universal computation: the TM-to-Cellaria translation preserves both time (one tick per

step) and space (linear in tape size). This suggests that locality imposes no asymptotic computational penalty in space or time for the class of Turing machines.

1.8 References

1. Minsky, M. L. (1961). “Recursive Unsolvability of Post’s Problem of Tag and Other Topics in Theory of Turing Machines.” *Annals of Mathematics*, 74(3), 437–455.
2. Cocke, J., & Minsky, M. (1964). “Universality of Tag Systems with P=2.” *Journal of the ACM*, 11(1), 15–20.
3. Lafont, Y. (1990). “Interaction Nets.” *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 95–108.
4. Păun, G. (1998). “Computing with Membranes.” *Journal of Computer and System Sciences*, 61(1), 108–143.
5. Berry, G., & Boudol, G. (1992). “The Chemical Abstract Machine.” *Theoretical Computer Science*, 96(1), 217–248.
6. Turing, A. M. (1936). “On Computable Numbers, with an Application to the Entscheidungsproblem.” *Proceedings of the London Mathematical Society*, s2-42(1), 230–265.
7. Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media. [On cellular automata and universality.]
8. Cook, M. (2004). “Universality in Elementary Cellular Automata.” *Complex Systems*, 15(1), 1–40.
9. Giavitto, J.-L., & Michel, O. (2002). “The Topological Structures of Membrane Computing.” *Fundamenta Informaticae*, 49(1-3), 123–145.
10. Abelson, H., et al. (2000). “Amorphous Computing.” *Communications of the ACM*, 43(5), 74–82.
11. Plump, D. (2012). “The Graph Programming Language GP 2.” *EATCS Bulletin*, 108, 49–68.

1.9 Appendix A: Full Example — Turing Machine Bit Inversion

1.9.1 A.1. Configuration

The complete configuration `configs/turing.yaml`:

```yaml
grid:
  width: 16
  height: 1
  default_cell_type: 0

  initial_cells:
    - coord: [0, 0]
      type: 10          # head marker (state Q□)
    - coord: [1, 0]
```

```
      type: 1          # tape symbol 1
    - coord: [2, 0]
      type: 2          # tape symbol 0
    - coord: [3, 0]
      type: 1          # tape symbol 1
    - coord: [4, 0]
      type: 1          # tape symbol 1
    - coord: [5, 0]
      type: 2          # tape symbol 0

  boundaries: []

rules:
  # (Q□, 0) → (Q□, 1), move right
  - id: [10, 2]        # 10=head, 2=enc(0)
    priority: 10
    shifts:
      - group:
          - direction: east
            steps: 1
    changes:
      - [-1, 0, 1]     # write 1 (enc(1)) at vacated position

  # (Q□, 1) → (Q□, 0), move right
  - id: [10, 1]        # 10=head, 1=enc(1)
    priority: 10
    shifts:
      - group:
          - direction: east
            steps: 1
    changes:
      - [-1, 0, 2]     # write 2 (enc(0)) at vacated position
```

1.9.2 A.2. Encoding

TM component	Cellaria type
Head in state Q□	10
Tape symbol 1	1
Tape symbol 0	2
Blank (end of tape)	0

The inversion encoding (type 1 = symbol 1, type 2 = symbol 0) is a design choice for readability. The actual mapping is irrelevant to the proof.

1.9.3 A.3. Trace (simulation output)

Input: `1 0 1 1 0` (binary `10110`)

```
Tick 0: [10, 1, 2, 1, 1, 2, 0, 0, ...]   head at pos 0
Tick 1: [2, 10, 2, 1, 1, 2, 0, 0, ...]    write 0, move right
Tick 2: [2, 1, 10, 1, 1, 2, 0, 0, ...]    write 1, move right
Tick 3: [2, 1, 2, 10, 1, 2, 0, 0, ...]    write 0, move right
Tick 4: [2, 1, 2, 2, 10, 2, 0, 0, ...]    write 0, move right
Tick 5: [2, 1, 2, 2, 1, 10, 0, 0, ...]    write 1, move right
Tick 6: [2, 1, 2, 2, 1, 10, 0, 0, ...]   halt (no match for [10, 0])
```

At tick 6, the head is at position 5. The pattern `[10, 0]` (head followed by blank) does not match any rule (we only have `[10, 1]` and `[10, 2]`). The system enters a stable state with zero matches.

Output: `2 1 2 2 1` = `0 1 0 0 1` (binary `01001` = bit-inverted `10110`).

The head never turns around; it traverses the tape once, inverting each bit, and halts when it reaches the first blank cell. This simulates a TM with a single state and a right-moving head.

1.9.4 A.4. Decoding

Cellaria output	Decoded value
2	0
1	1
2	0
2	0
1	1

Result: `01001` = bit-inverted input `10110` □

Cellaria source code and configurations: https://github.com/PixelDeus/Cellaria